

Pre-practica Project Documentation

IoT Telemetry Platform with Local Resilience, Blockchain Integrity Verification and Mobile Monitoring

Dimitrie Constantin

July 2026

Faculty of Automatic Control and Computer Science
National University of Science and Technology POLITEHNICA Bucharest

Contents

1	Introduction	3
2	Project Objectives and Requirements	3
2.1	Non-functional Requirements	4
2.2	Project Scope	4
3	Technologies Used	4
3.1	Docker	5
3.2	MQTT	5
3.3	Node-RED	5
3.4	InfluxDB	5
3.5	Grafana	5
3.6	Python	5
3.7	systemd	6
3.8	Ganache	6
3.9	Solidity	6
3.10	Web3.py	6
3.11	Flutter	6
3.12	Eclipse Arrowhead	7
4	System Architecture	7
4.1	General Data Flow	7
4.2	Local Resilience Architecture	7
4.3	Blockchain Integrity Architecture	8
4.4	Mobile Access Architecture	8
4.5	Arrowhead-Oriented Architecture	8
4.6	Overall Architecture Summary	8
5	Implementation	9
5.1	Phase 1 – IoT Telemetry Pipeline	9
5.2	Phase 2 – Local Storage and Resynchronization	10
5.3	Phase 3 – Blockchain Data Integrity	10
5.4	Phase 4 – Arrowhead Integration Analysis	10
5.5	Phase 5 – Mobile Application	11
5.6	Implementation Summary	11
6	Running the Project	12
6.1	Prerequisites	12
6.2	Cloning the Repository	12
6.3	Starting the Docker Services	13
6.4	Deploying the Blockchain Contract	13
6.5	Starting the Background Services	13
6.6	Verifying Blockchain Integrity	13
6.7	Stopping the Project	14
6.8	Typical Execution Flow	14
6.9	Running the Flutter Mobile Application	14

7	Testing and Validation	14
7.1	Telemetry Pipeline Validation	15
7.2	Local Storage and Synchronization Validation	15
7.3	Blockchain Anchoring Validation	16
7.4	Blockchain Integrity Verification	17
7.5	HTTP Provider and Mobile Application Validation	17
7.6	Arrowhead Validation	20
7.7	Validation Summary	21
8	Results	21
8.1	Telemetry and Visualization Results	21
8.2	Local Resilience Results	22
8.3	Blockchain Integrity Results	22
8.4	Backend Service Results	22
8.5	Mobile Application Results	22
8.6	Arrowhead Integration Result	23
8.7	Final Result Summary	23
9	Limitations and Future Work	23
9.1	Limitations	24
9.1.1	Simulated IoT Device	24
9.1.2	Local Development Blockchain	24
9.1.3	Incomplete Arrowhead Core Integration	24
9.1.4	Limited Security Mechanisms	24
9.1.5	Limited Mobile Features	24
9.2	Future Work	25
9.3	Future Architecture Improvement	25
9.4	Summary	25
10	Conclusion	26
11	References	26

1 Introduction

The purpose of this project was to design and implement a **modular IoT telemetry platform** capable of collecting, storing, visualizing and validating sensor data. The system was developed during the pre-practica period and focuses on building a complete end-to-end data flow, starting from temperature data generation and ending with visualization, persistence, integrity verification and mobile access.

The project simulates an IoT device that periodically generates temperature readings. These readings are transmitted using the **MQTT protocol**, processed through **Node-RED**, stored in **InfluxDB** and displayed in Grafana. In order to improve reliability, the system also includes a **local storage and synchronization mechanism**, which allows data to be preserved and later sent to the database after temporary service or network interruptions.

Another important part of the project is **data integrity verification**. For this purpose, batches of sensor readings are hashed using the Keccak-256 algorithm, and the resulting hashes are stored on a local **blockchain** deployed with Ganache. This makes it possible to verify whether the locally stored data has been modified after it was recorded.

The project also includes a **mobile application** developed with **Flutter**, which allows the user to monitor live temperature values and view historical readings. In addition, Eclipse Arrowhead was analyzed as a possible solution for service discovery, orchestration and authorization. Although the full Arrowhead Core integration was not deployed, the provider service and the intended integration architecture were documented.

Overall, the project demonstrates how several modern technologies can be combined into a functional IoT monitoring system with local resilience, visualization, blockchain-based integrity verification and mobile access.

2 Project Objectives and Requirements

The main objective of this project was to design and implement a complete IoT telemetry platform for temperature monitoring. The platform had to collect sensor data, transmit it through an MQTT broker, process and store it, visualize it in real time, provide local resilience in case of service interruptions, verify data integrity using blockchain anchoring, and expose the information through a mobile application. The project was divided into several technical objectives:

- to simulate an IoT temperature sensor and generate periodic measurements;
- to transmit sensor readings using the MQTT publish/subscribe protocol;
- to process incoming data using Node-RED flows;
- to store time-series data in InfluxDB;
- to visualize live temperature values using Grafana dashboards;
- to implement local storage in order to avoid data loss during temporary failures;
- to automatically synchronize locally stored data after the database becomes available again;

- to implement blockchain-based integrity verification by anchoring hashes of data batches;
- to develop a mobile application for live and historical temperature monitoring;
- to analyze the possibility of integrating the system with Eclipse Arrowhead for service discovery and orchestration.

2.1 Non-functional Requirements

The non-functional requirements describe the quality and operational aspects of the system:

- The system should be modular, with separate components for data acquisition, processing, storage, visualization, synchronization, blockchain verification and mobile access.
- The main services should be easy to start, stop and restart.
- Docker should be used for containerized services such as InfluxDB, Node-RED, Grafana and Ganache.
- Background services should be managed using systemd in order to run automatically and recover after restarts.
- The system should be resilient to temporary interruptions of the database or network.
- The architecture should allow future extension with a physical IoT device, additional sensors, notifications and Arrowhead-based service discovery.

2.2 Project Scope

The implemented system focuses on a practical prototype rather than a production deployment. The IoT device is simulated using Node-RED, while the blockchain component runs on a local Ganache development network. The Arrowhead integration was analyzed and prepared at the provider level, but the complete Arrowhead Core deployment was not included in the final implementation due to version incompatibilities and configuration complexity.

3 Technologies Used

This section presents the main technologies used in the implementation of the IoT telemetry platform. Each technology was selected according to its role in the system architecture, from data generation and communication to storage, visualization, resilience, blockchain integrity verification and mobile access.

3.1 Docker

Docker was used to run several project components in isolated containers. This made the development environment easier to configure and reproduce. In this project, Docker was used for services such as InfluxDB, Node-RED, Grafana and Ganache.

Docker Compose was also used to start and manage multiple services together. This allowed the main backend stack to be launched with a single command and simplified the communication between containers.

3.2 MQTT

MQTT is a lightweight publish/subscribe communication protocol commonly used in IoT systems. In this project, MQTT was used to transmit temperature readings from the simulated IoT device to the data processing layer.

The sensor simulator published messages to a specific MQTT topic, while Node-RED subscribed to that topic and processed the incoming data.

3.3 Node-RED

Node-RED was used as the main flow-based data processing tool. It allowed the creation of visual flows for generating, receiving, parsing and forwarding sensor data.

In the project, Node-RED had two important roles. First, it was used to simulate an IoT temperature sensor. Second, it was used to receive MQTT messages, process them and send them to the storage layer.

3.4 InfluxDB

InfluxDB is a time-series database designed for storing timestamped data. It was used to store temperature readings collected from the IoT pipeline.

The data was stored using a measurement dedicated to sensor readings, with the temperature value saved as a field and the device name saved as a tag. This structure made it possible to query and visualize the evolution of temperature over time.

3.5 Grafana

Grafana was used for data visualization. It was connected to InfluxDB and configured to display live temperature values in a dashboard.

Grafana provided a visual way to validate that the telemetry pipeline was working correctly, because new sensor readings could be observed in real time.

3.6 Python

Python was used for several backend scripts in the project. It was chosen because it is simple, flexible and has good support for HTTP communication, file processing and blockchain interaction.

The main Python components were:

- the synchronization script, used to transfer locally stored readings to InfluxDB;

- the blockchain anchoring script, used to hash batches of readings and send them to the smart contract;
- the verification script, used to compare local hashes with blockchain hashes;
- the HTTP provider, used to expose temperature data to the mobile application.

3.7 systemd

systemd was used to run important backend scripts as Linux services. This allowed the synchronization service, blockchain anchoring service and HTTP provider service to run continuously in the background.

Using systemd also improved reliability, because the services could be started automatically after reboot and restarted if they failed.

3.8 Ganache

Ganache was used as a local Ethereum blockchain for development and testing. It provided a local blockchain environment where smart contracts could be deployed and tested without using real cryptocurrency or a public blockchain network.

In this project, Ganache was used to demonstrate blockchain-based data integrity verification.

3.9 Solidity

Solidity was used to write the smart contract responsible for storing hashes of sensor data batches. The smart contract did not store the raw sensor data. Instead, it stored only the hash of each batch, together with blockchain-level information such as the transaction timestamp.

This approach reduced the amount of data stored on-chain while still allowing integrity verification.

3.10 Web3.py

Web3.py was used to interact with the local Ethereum blockchain from Python. It allowed the Python scripts to deploy the smart contract, call contract functions and read stored hashes from the blockchain.

In the project, Web3.py connected the blockchain logic with the local sensor data stored in JSONL format.

3.11 Flutter

Flutter was used to develop the mobile application. The application communicates with the backend HTTP provider and displays live and historical temperature values.

The mobile application provides a user-friendly interface for monitoring the telemetry data without accessing Grafana or the terminal.

3.12 Eclipse Arrowhead

Eclipse Arrowhead was analyzed as a service-oriented framework for IoT and industrial systems. Its purpose was to provide service discovery, orchestration and authorization.

In the intended architecture, the temperature provider would register itself in the Arrowhead Service Registry, and a consumer such as the mobile application would discover it through the Orchestrator. However, the complete Arrowhead Core integration was not deployed due to version incompatibilities and configuration complexity. The project therefore kept the HTTP provider functional and documented the Arrowhead integration as future work.

4 System Architecture

The system was designed as a modular IoT telemetry platform composed of several independent components. Each component has a clearly defined role in the data flow, from data generation to storage, visualization, integrity verification, and mobile access.

The general architecture follows an end-to-end telemetry pipeline. Temperature readings are generated by a simulated IoT device, transmitted through MQTT, processed by Node-RED, stored locally and in InfluxDB, visualized in Grafana, anchored on a local blockchain and exposed to a mobile application through an HTTP provider.

4.1 General Data Flow

The main data flow of the system is the following:

Sensor Simulator → MQTT Broker → Node-RED → JSONL Storage → InfluxDB
→ Grafana

The sensor simulator periodically generates temperature readings. These readings are published to an MQTT topic. Node-RED subscribes to this topic, receives the messages, parses the JSON payload and forwards the processed data to the storage layer.

InfluxDB stores the readings as time-series data, while Grafana connects to InfluxDB and displays the temperature values in real time. This represents the core telemetry pipeline of the project.

4.2 Local Resilience Architecture

In order to reduce the risk of data loss, the system also includes a local persistence mechanism. Before or during synchronization with InfluxDB, the readings are stored in a local JSONL file. Each line in this file represents one sensor reading.

A Python synchronization service reads the local file and sends unsynchronized readings to InfluxDB. The service uses a checkpoint file to remember how many lines have already been successfully synchronized. If InfluxDB is temporarily unavailable, the data remains stored locally and is synchronized later when the database becomes available again.

The resilience flow can be represented as:

Sensor Data → Local JSONL File → sync.py → InfluxDB

This approach improves reliability because temporary database or network interruptions do not immediately result in data loss.

4.3 Blockchain Integrity Architecture

The blockchain component was added to provide data integrity verification. The raw sensor data is not stored directly on the blockchain. Instead, the readings stored in the JSONL file are grouped into batches of 50 entries. For each batch, a Keccak-256 hash is computed.

The resulting hash is then stored in a smart contract deployed on a local Ganache blockchain. Later, the verification script can recompute the hash from the local data and compare it with the hash stored on-chain.

The blockchain integrity flow is:

JSONL Data → Batch of 50 Readings → Keccak-256 Hash → Smart Contract

This architecture allows the system to prove whether a batch of sensor readings has been modified after being anchored.

4.4 Mobile Access Architecture

The mobile application does not connect directly to InfluxDB. Instead, a Python HTTP provider was implemented as an intermediate backend service. This provider reads live and historical temperature values from InfluxDB and exposes them through HTTP endpoints.

The mobile access flow is:

InfluxDB → HTTP Provider → Flutter Mobile Application

The Flutter application periodically requests the latest temperature value and also retrieves historical readings. This provides a user-friendly interface for monitoring the IoT data without using terminal commands or Grafana directly.

4.5 Arrowhead-Oriented Architecture

Eclipse Arrowhead was analyzed as a possible service discovery and orchestration layer. In the intended architecture, the HTTP temperature provider would register itself in the Arrowhead Service Registry. A consumer, such as the mobile application, would then request the service through the Arrowhead Orchestrator instead of using a hardcoded IP address.

The intended Arrowhead-based flow was:

Provider → Service Registry

Consumer → Orchestrator → Service Registry / Authorization → Provider

In the final implementation, the provider remained functional as a direct HTTP service. The full Arrowhead Core deployment was not completed because the reference implementation used the Arrowhead 4.x API, while the readily available Docker images were based on Arrowhead 5.x and were not directly compatible.

4.6 Overall Architecture Summary

The complete implemented architecture can be summarized as follows:

This modular architecture made it possible to implement and test each component independently, while still integrating them into a complete IoT monitoring platform.

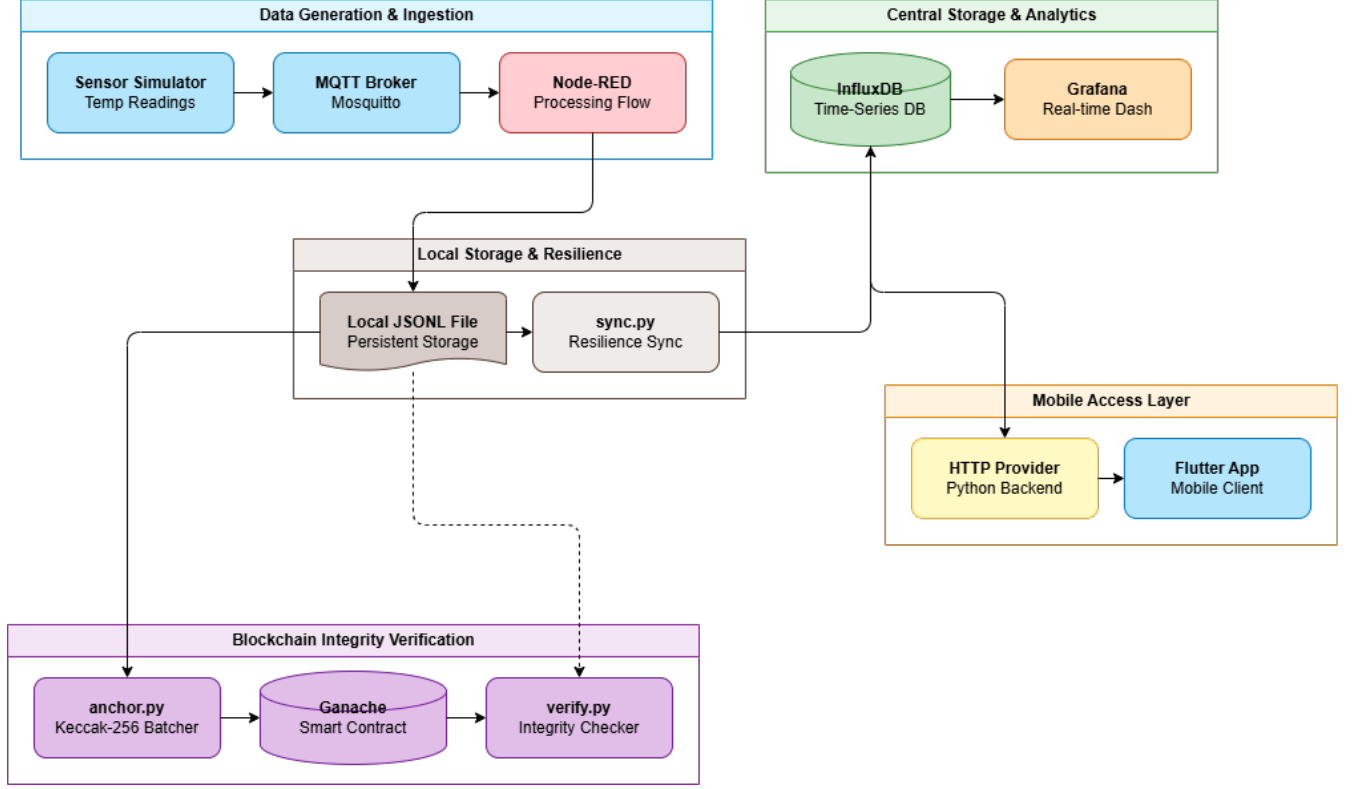


Figure 1: Overall system architecture of the IoT telemetry platform

5 Implementation

This section describes the practical implementation of the IoT telemetry platform. The project was implemented in several phases, starting from the basic telemetry pipeline and continuing with local resilience, blockchain integrity verification, Arrowhead integration analysis and mobile access.

5.1 Phase 1 – IoT Telemetry Pipeline

The first phase focused on implementing the core IoT telemetry pipeline. A simulated temperature sensor was created using Node-RED. The simulator periodically generated temperature readings and published them to an MQTT topic.

The MQTT broker was used as the communication layer between the simulated sensor and the processing flow. Node-RED subscribed to the MQTT topic, received the JSON messages, parsed the payload and prepared the data for storage.

The processed readings were then written to InfluxDB, which was used as the time-series database of the project. Grafana was connected to InfluxDB and used to create a real-time dashboard for visualizing the temperature values.

The implemented data flow for this phase was:

Sensor Simulator → MQTT Broker → Node-RED → InfluxDB → Grafana

This phase provided the basic functional pipeline of the project and confirmed that generated IoT data could be transmitted, processed, stored and visualized.

5.2 Phase 2 – Local Storage and Resynchronization

The second phase added local resilience to the system. The purpose of this phase was to reduce the risk of data loss during temporary service or network interruptions.

A local JSONL file was used as persistent storage. Each generated sensor reading was stored as a separate line in this file. This format was simple, easy to append to, and suitable for later synchronization.

A Python script named `sync.py` was implemented to synchronize the local data with InfluxDB. The script reads the JSONL file, detects which lines have not yet been synchronized and sends them to the database. In order to avoid sending the same data multiple times, the script uses a checkpoint file that stores the number of lines already processed.

The synchronization mechanism follows this flow:

Local JSONL File → `sync.py` → InfluxDB

The script was also configured as a systemd service, allowing it to run continuously in the background and start automatically after system reboot. This improved the reliability of the platform and made the synchronization process independent of manual execution.

5.3 Phase 3 – Blockchain Data Integrity

The third phase implemented blockchain-based integrity verification. The goal was not to store all sensor readings directly on the blockchain, but to store compact cryptographic proofs of the data.

The readings stored in the local JSONL file were grouped into batches of 50 entries. For each batch, a Keccak-256 hash was computed. This hash uniquely represents the content of the batch. If any reading in the batch is modified, the recalculated hash will be different.

A Solidity smart contract was created and deployed on a local Ganache blockchain. The contract stores the hash of each batch, using the batch identifier as an index. The Python script `anchor.py` reads the local data, calculates the batch hashes and sends them to the smart contract using Web3.py.

The implemented blockchain flow was:

JSONL Data → Batch of 50 Readings → Keccak-256 Hash → Ganache Smart Contract

A separate script, `verify.py`, was implemented to validate data integrity. It recalculates the hash of a selected batch from the local JSONL file and compares it with the hash stored on the blockchain. If both hashes match, the data batch is considered unchanged.

The blockchain anchoring script was also configured as a systemd service, so new batches could be anchored automatically while the system was running.

5.4 Phase 4 – Arrowhead Integration Analysis

The fourth phase focused on analyzing the integration of the system with Eclipse Arrowhead. Arrowhead was considered as a possible service discovery and orchestration layer for the project.

The intended architecture was that the temperature provider would register itself in the Arrowhead Service Registry. A consumer, such as the mobile application, would then

request the temperature service through the Orchestrator instead of using a hardcoded IP address. The Authorization system would control whether the consumer was allowed to access the provider.

The intended Arrowhead flow was:

Provider → Service Registry

Consumer → Orchestrator → Authorization / Service Registry → Provider

In the final implementation, the complete Arrowhead Core deployment was not finalized. The main reason was the incompatibility between the Arrowhead 4.x API used by the reference implementation and the readily available Docker images based on Arrowhead 5.x. The compatible 4.6.2 version required manual compilation and additional configuration for the database, certificates and core services.

Therefore, the project kept the HTTP temperature provider functional and documented the Arrowhead-based architecture as future work.

5.5 Phase 5 – Mobile Application

The fifth phase implemented mobile access to the telemetry data. A Flutter application was developed and tested on an Android device.

The mobile application communicates with a Python HTTP provider. The provider reads the latest and historical temperature values from InfluxDB and exposes them through HTTP endpoints. This avoids direct communication between the mobile application and the database.

The mobile access flow was:

InfluxDB → HTTP Provider → Flutter Mobile Application

The application displays live temperature values and historical readings. This provides a simple user interface for monitoring the system without using Grafana or terminal commands.

The HTTP provider was also configured as a systemd service, allowing it to run continuously in the background and serve requests from the mobile application.

5.6 Implementation Summary

At the end of the implementation phase, the system included the following functional components:

- a Node-RED temperature sensor simulator;
- MQTT-based communication;
- Node-RED processing flows;
- InfluxDB time-series storage;
- Grafana visualization;
- local JSONL persistence;

- automatic synchronization with InfluxDB;
- blockchain anchoring using Ganache, Solidity and Web3.py;
- blockchain integrity verification using `verify.py`;
- a Python HTTP provider;
- a Flutter mobile application for live and historical monitoring;
- systemd services for background execution.

The implementation resulted in a modular IoT telemetry prototype with data acquisition, storage, visualization, resilience, blockchain-based integrity verification and mobile access.

6 Running the Project

The complete source code and configuration files are available in the project GitHub repository:

```
https://github.com/DimiConstantin/beia-iot-internship
```

The repository contains the Docker configuration, Node-RED flows, Python backend scripts, blockchain smart contract, Web3.py scripts, systemd service files and Flutter mobile application code.

6.1 Prerequisites

Before running the project, the following tools should be installed:

- Docker and Docker Compose;
- Python 3;
- pip packages required by the backend scripts;
- systemd, for running the backend services;
- Flutter, if the mobile application is built locally.

6.2 Cloning the Repository

The project can be downloaded from GitHub using:

```
git clone https://github.com/DimiConstantin/beia-iot-internship.git
cd beia-iot-internship
```

6.3 Starting the Docker Services

The containerized services can be started using the Makefile:

```
make up
```

This command starts the Docker Compose stack, including services such as InfluxDB, Node-RED, Grafana and Ganache.

The running containers can be checked with:

```
make ps
```

The logs can be monitored using:

```
make logs
```

6.4 Deploying the Blockchain Contract

After the Docker services are running, the blockchain smart contract can be deployed using:

```
make deploy
```

This command runs the deployment script from the blockchain folder and generates the local deployment information required by the anchoring and verification scripts.

If Ganache is restarted or reset, the contract must be deployed again, because Ganache is used as a local development blockchain.

6.5 Starting the Background Services

The backend scripts are managed as systemd services. They can be started with:

```
make services
```

This starts the synchronization service, the blockchain anchoring service and the HTTP provider service.

Their status can be checked using:

```
make status
```

The expected result is that the services are shown as active and running.

6.6 Verifying Blockchain Integrity

After sensor data has been generated and at least one batch has been anchored, the integrity verification script can be executed with:

```
make verify
```

This command verifies batch 0 by recalculating the local hash and comparing it with the hash stored in the smart contract.

6.7 Stopping the Project

The background services can be stopped with:

```
make stop-services
```

The Docker services can be stopped with:

```
make down
```

6.8 Typical Execution Flow

A typical execution flow for running the project is:

```
make up
```

```
make deploy
```

```
make services
```

```
make status
```

```
make verify
```

This sequence starts the infrastructure, deploys the blockchain smart contract, starts the backend services and verifies the integrity mechanism.

6.9 Running the Flutter Mobile Application

The Flutter mobile application source code is included in the same GitHub repository, inside the `mobile/flutter_app` directory.

The application was developed and tested in the Windows environment, while the backend services were executed in Linux/WSL. To run the mobile application, the user should open the Flutter project from the `mobile/flutter_app` directory using Android Studio or Visual Studio Code on Windows.

The required commands are:

```
flutter pub get
```

```
flutter run
```

The mobile application must be configured with the IP address of the HTTP provider. During testing, this address was not always persistent, therefore future versions should replace the hardcoded address with a configurable setting or service discovery mechanism.

7 Testing and Validation

This section presents the tests performed to validate the main components of the IoT telemetry platform. Each test describes the validation method, the expected result and the evidence collected during execution, such as terminal logs, service status outputs, dashboards and mobile application screenshots.

7.1 Telemetry Pipeline Validation

The first validation step was to test the complete telemetry pipeline from data generation to visualization. The Node-RED simulator was configured to generate periodic temperature readings and publish them to the MQTT broker. A second Node-RED flow subscribed to the MQTT topic, parsed the JSON payload and forwarded the processed data to InfluxDB.

The validation method consisted of checking:

- that messages were generated periodically by the simulator;
- that Node-RED received the MQTT messages;
- that the values were stored in InfluxDB;
- that Grafana displayed the live temperature data.

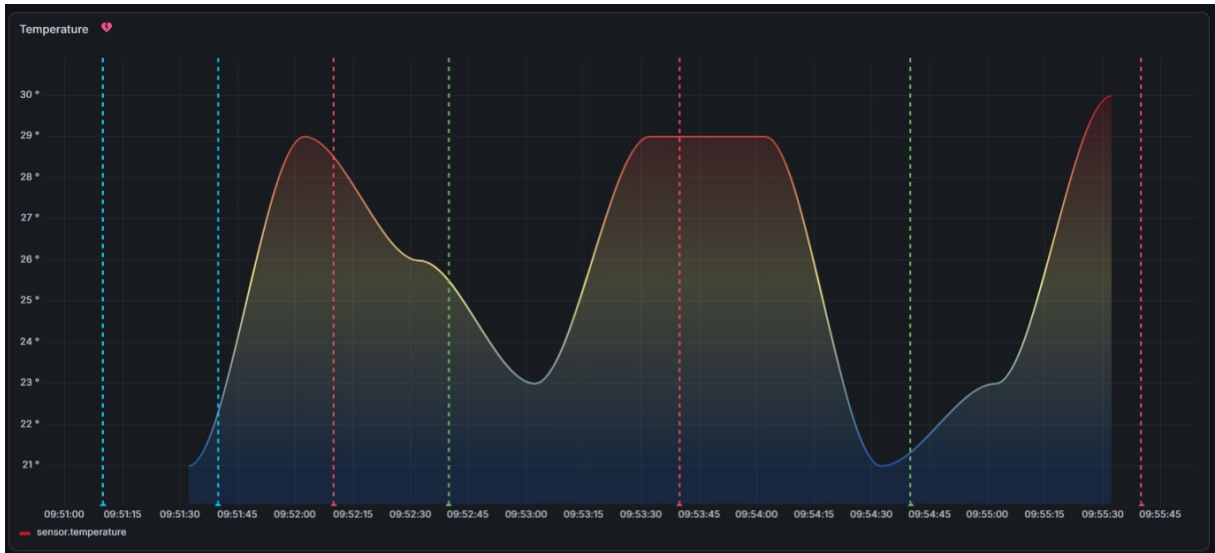


Figure 2: Temperature values in Grafana

The successful display of live temperature values in Grafana confirmed that the main telemetry pipeline was functional.

7.2 Local Storage and Synchronization Validation

The local resilience mechanism was validated by checking that sensor readings were written to the local JSONL file and then synchronized with InfluxDB by the Python synchronization script.

The validation method consisted of verifying the local storage file, checking the synchronization checkpoint and monitoring the systemd service responsible for running the synchronization process in the background.

The service status was checked using:

```
systemctl status iot-sync.service
```


The test was successful when the service was shown as active (running) and the logs confirmed that readings were synchronized with InfluxDB.

```
dimi@Asus23:~/practica/proiect/local-sync$ systemctl status iot-sync.service
● iot-sync.service - IoT local-to-cloud sync service
   Loaded: loaded (/etc/systemd/system/iot-sync.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-07-08 09:50:58 EEST; 55min ago
     Main PID: 1464 (python3)
        Tasks: 1 (limit: 18694)
       Memory: 18.1M
          CPU: 1.483s
      CGroup: /system.slice/iot-sync.service
              └─1464 /usr/bin/python3 /home/dimi/practica/proiect/local-sync/sync.py

Jul 08 10:41:34 Asus23 python3[1464]: 10:41:34 [INFO] sincronised 1 readings from 4484 in total
Jul 08 10:42:04 Asus23 python3[1464]: 10:42:04 [INFO] sincronised 1 readings from 4485 in total
Jul 08 10:42:34 Asus23 python3[1464]: 10:42:34 [INFO] sincronised 1 readings from 4486 in total
Jul 08 10:43:04 Asus23 python3[1464]: 10:43:04 [INFO] sincronised 1 readings from 4487 in total
Jul 08 10:43:34 Asus23 python3[1464]: 10:43:34 [INFO] sincronised 1 readings from 4488 in total
Jul 08 10:44:04 Asus23 python3[1464]: 10:44:04 [INFO] sincronised 1 readings from 4489 in total
Jul 08 10:44:34 Asus23 python3[1464]: 10:44:34 [INFO] sincronised 1 readings from 4490 in total
Jul 08 10:45:04 Asus23 python3[1464]: 10:45:04 [INFO] sincronised 1 readings from 4491 in total
Jul 08 10:45:34 Asus23 python3[1464]: 10:45:34 [INFO] sincronised 1 readings from 4492 in total
Jul 08 10:46:04 Asus23 python3[1464]: 10:46:04 [INFO] sincronised 1 readings from 4493 in total
```

Figure 3: Synchronization service running as a systemd background service

7.3 Blockchain Anchoring Validation

The blockchain anchoring mechanism was validated by running the `anchor.py` script as a systemd service. The script grouped sensor readings into batches of 50 entries, computed a Keccak-256 hash for each batch and stored the hash in the smart contract deployed on Ganache.

The validation method consisted of checking the systemd service logs and confirming that new batches were periodically anchored on the blockchain.

The service status was checked using:

```
systemctl status iot-anchor.service
```

The logs confirmed that multiple batches were successfully anchored.

```
dimi@Asus23:~/practica/proiect/local-sync$ systemctl status iot-anchor.service
● iot-anchor.service - IoT blockchain anchoring service
   Loaded: loaded (/etc/systemd/system/iot-anchor.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-07-08 09:50:58 EEST; 1h 12min ago
     Main PID: 1457 (python3)
        Tasks: 1 (limit: 18694)
       Memory: 48.5M
          CPU: 2.110s
      CGroup: /system.slice/iot-anchor.service
              └─1457 /usr/bin/python3 /home/dimi/practica/proiect/blockchain/anchor.py

Jul 08 09:50:58 Asus23 systemd[1]: Started IoT blockchain anchoring service.
Jul 08 09:51:00 Asus23 python3[1457]: 09:51:00 [INFO] Serviciu de anchoring pornit (lot=50, interval=15s)
Jul 08 09:59:46 Asus23 python3[1457]: 09:59:46 [INFO] Lot 87 ancorat: 7c8937e0cc4ee390c44376a9e19bf3c39e63116648e80143d5
Jul 08 10:24:33 Asus23 python3[1457]: 10:24:33 [INFO] Lot 88 ancorat: 9d78b812e739d5f15edbe72f7158fafaed217f2866cc0605a5
Jul 08 10:49:35 Asus23 python3[1457]: 10:49:35 [INFO] Lot 89 ancorat: f36972433c827104275018e57687d30a7d3a8fbfc1d5d98a75
```

Figure 4: Blockchain anchoring service running continuously and anchoring new data batches

7.4 Blockchain Integrity Verification

The integrity verification was performed using the `verify.py` script. The script recalculates the Keccak-256 hash of a selected local data batch and compares it with the hash stored in the smart contract.

The validation command was:

```
python3 verify.py 0
```

If the locally computed hash matches the hash stored on the blockchain, the selected data batch is considered unchanged. This confirms that the anchoring mechanism can be used to detect later modifications of the stored sensor data.

```
dimi@Asus23:~/practica/proiect/blockchain$ python3 verify.py 0
Lot 0
Hash local (recalculat): 1dad846d724fb845a02e5491caab23341856a8c6a69a0897215c12b741bb2ff6
Hash pe blockchain:      1dad846d724fb845a02e5491caab23341856a8c6a69a0897215c12b741bb2ff6
✓ INTACT - datele nu au fost modificate
```

Figure 5: Blockchain verification showing matching local and on-chain hashes

7.5 HTTP Provider and Mobile Application Validation

The HTTP provider was validated by checking that it was running as a systemd service and that it received requests from the mobile application. The provider retrieves live and historical temperature data from InfluxDB and exposes it through HTTP endpoints.

The validation method consisted of checking the provider logs and verifying that the Flutter application displayed the received data correctly.

During validation, one practical issue was observed: the backend IP address used by the mobile application was not always persistent. After network changes or restarts, the Flutter application could lose connection to the HTTP provider if the old IP address was still configured. This showed the limitation of using a hardcoded provider address and motivated the need for a configurable backend address or a future service discovery mechanism such as Arrowhead.

```
dimi@Asus23:~/practica/proiect/blockchain$ systemctl status iot-provider.service
● iot-provider.service - IoT Arrowhead provider (HTTP temperature service)
   Loaded: loaded (/etc/systemd/system/iot-provider.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-07-08 09:50:58 EEST; 1h 40min ago
     Main PID: 1463 (python3)
        Tasks: 1 (limit: 18694)
       Memory: 23.3M
          CPU: 1.239s
    CGroup: /system.slice/iot-provider.service
            └─1463 /usr/bin/python3 /home/dimi/practica/proiect/arrowhead/provider.py

Jul 08 09:50:58 Asus23 systemd[1]: Started IoT Arrowhead provider (HTTP temperature service).
Jul 08 09:50:58 Asus23 python3[1463]: 09:50:58 [INFO] Provider pornit pe portul 5001
Jul 08 09:50:58 Asus23 python3[1463]: * Serving Flask app 'provider'
Jul 08 09:50:58 Asus23 python3[1463]: * Debug mode: off
Jul 08 09:50:58 Asus23 python3[1463]: 09:50:58 [INFO] WARNING: This is a development server. Do not use it in a product
Jul 08 09:50:58 Asus23 python3[1463]: * Running on all addresses (0.0.0.0)
Jul 08 09:50:58 Asus23 python3[1463]: * Running on http://127.0.0.1:5001
Jul 08 09:50:58 Asus23 python3[1463]: * Running on http://10.0.7.26:5001
Jul 08 09:50:58 Asus23 python3[1463]: 09:50:58 [INFO] Press CTRL+C to quit
```

Figure 6: HTTP provider service running and serving temperature requests

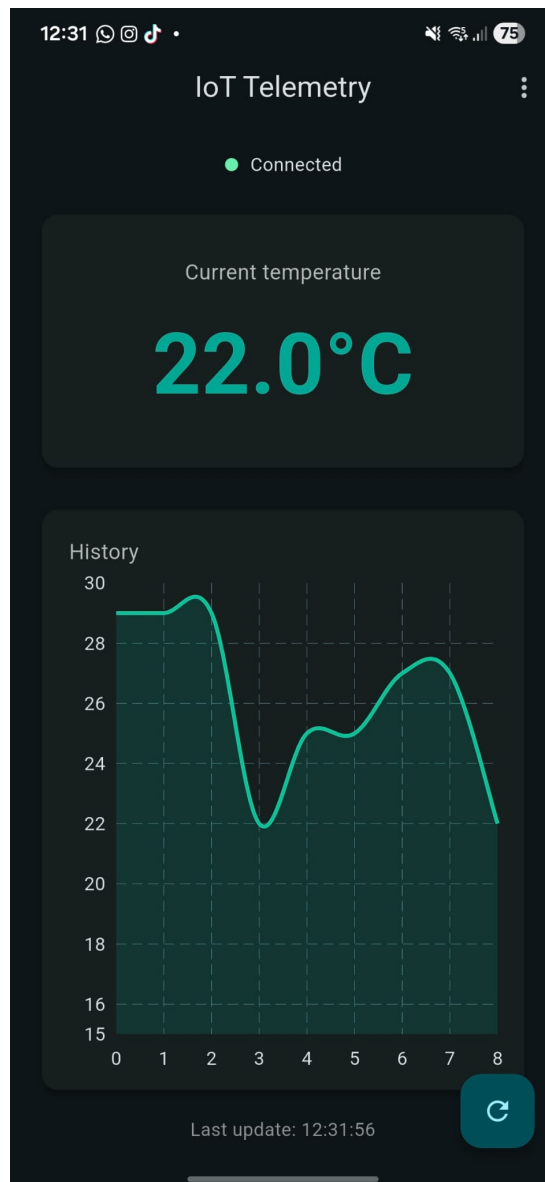


Figure 7: Flutter mobile application displaying live temperature data

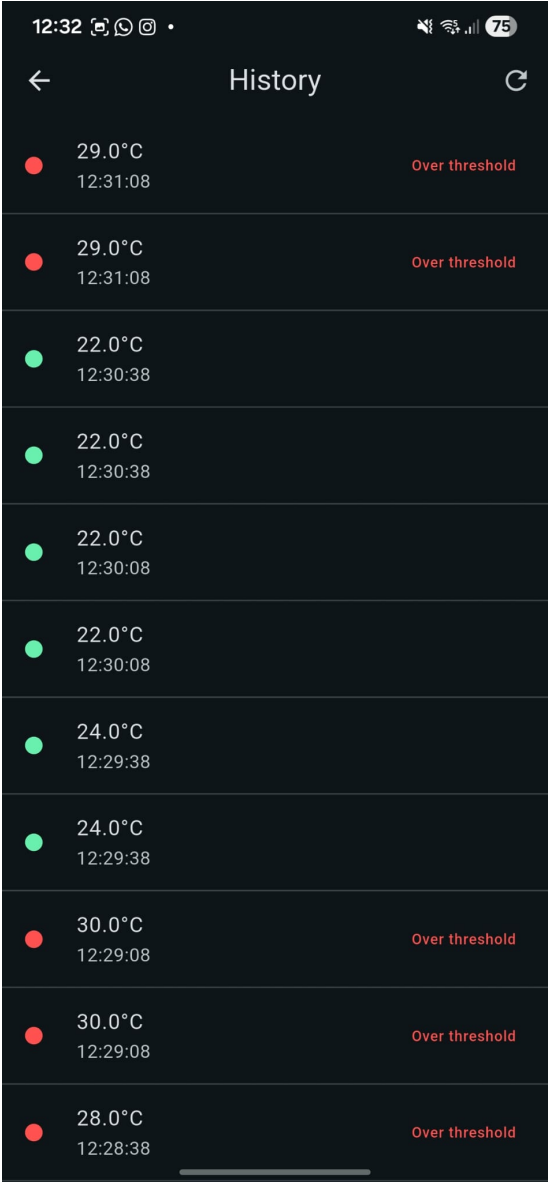


Figure 8: Flutter mobile application displaying historical temperature readings

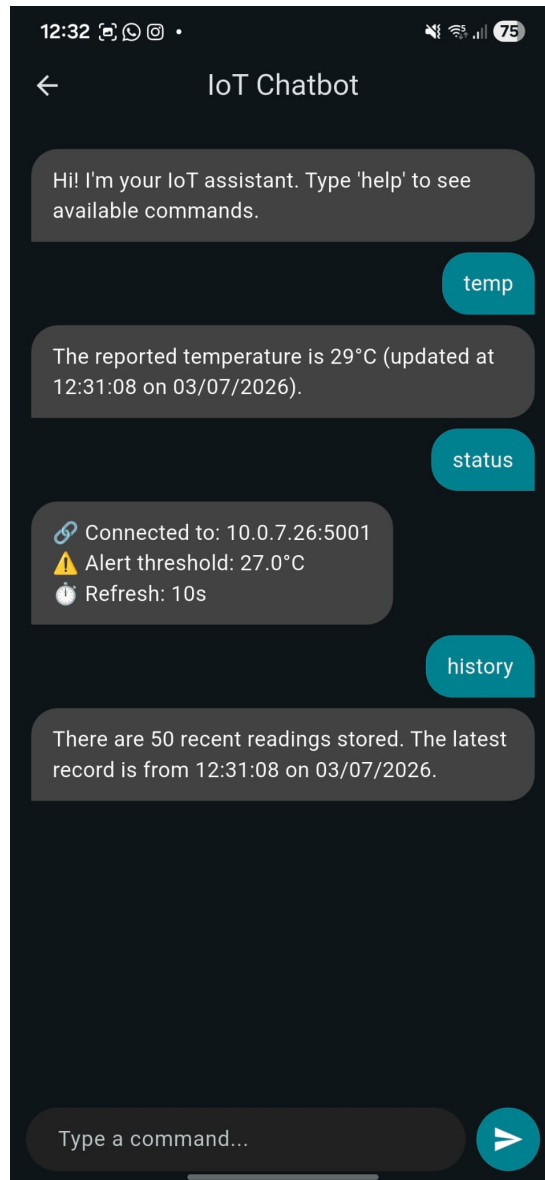


Figure 9: Flutter mobile application chatbot feature

7.6 Arrowhead Validation

The Arrowhead integration was validated at architectural and provider-preparation level. The intended role of Arrowhead was to provide service discovery, orchestration and authorization through the Service Registry, Orchestrator and Authorization systems.

A complete running validation of the Arrowhead Core was not performed. The reason was the incompatibility between the Arrowhead 4.x API used by the reference implementation and the readily available Arrowhead 5.x Docker images. The compatible 4.6.2 version required manual compilation and additional database and certificate configuration.

Therefore, this part was validated through architectural analysis and documentation of the intended integration flow. The HTTP provider remained functional as a direct backend service and can be adapted for future Arrowhead registration.

7.7 Validation Summary

The validation results are summarized below:

Component	Status	Validation Result
MQTT and Node-RED pipeline	Completed	Data was generated, transmitted and processed.
InfluxDB storage	Completed	Temperature readings were stored as time-series data.
Grafana dashboard	Completed	Live values were displayed in real time.
Local JSONL storage	Completed	Readings were stored locally.
Synchronization service	Completed	Local readings were synchronized with InfluxDB.
Blockchain anchoring	Completed	Hashes of data batches were stored on Ganache.
Blockchain verification	Completed	Local hashes matched the hashes stored on-chain.
HTTP provider	Completed	Live and historical data were exposed through HTTP.
Flutter mobile application	Completed	Live and historical values were displayed on Android.
Arrowhead Core integration	Partially completed	Architecture analyzed and documented; full Core deployment not finalized.

8 Results

The final result of the project is a functional IoT telemetry prototype that integrates data generation, communication, processing, storage, visualization, local resilience, blockchain-based integrity verification and mobile access.

The implemented system successfully demonstrates an end-to-end data flow from a simulated temperature sensor to a mobile application. Temperature readings are generated periodically, transmitted through MQTT, processed by Node-RED, stored in InfluxDB and visualized in Grafana. The system also stores readings locally in a JSONL file and uses a synchronization service to send unsynchronized data to InfluxDB.

8.1 Telemetry and Visualization Results

The first result is the successful implementation of the core telemetry pipeline. The Node-RED simulator generates temperature readings, while the processing flow receives the MQTT messages and writes them to InfluxDB.

Grafana was connected to InfluxDB and used to display the stored temperature values in real time. This confirms that the main telemetry pipeline is functional and that the system can collect, store and visualize time-series sensor data.

The achieved data flow is:

Sensor Simulator → MQTT Broker → Node-RED → InfluxDB → Grafana

8.2 Local Resilience Results

The second result is the implementation of a local resilience mechanism. Sensor readings are saved in a local JSONL file, which acts as persistent storage. This makes it possible to preserve data even when the database or another service is temporarily unavailable.

The synchronization script reads the local file and sends unsynchronized readings to InfluxDB. The checkpoint mechanism prevents duplicate synchronization by storing the number of already processed lines.

As a result, the system can recover from temporary interruptions and continue synchronizing data after the service becomes available again.

8.3 Blockchain Integrity Results

The third result is the implementation of blockchain-based data integrity verification. The system groups sensor readings into batches of 50 entries and computes a Keccak-256 hash for each batch.

The hashes are stored in a Solidity smart contract deployed on a local Ganache blockchain. The raw sensor data is not stored on-chain; only the cryptographic proof of each batch is stored. This reduces the amount of blockchain data while still allowing integrity verification.

The verification script recalculates the hash of a selected local batch and compares it with the hash stored in the smart contract. If the hashes match, the batch is considered unchanged.

This demonstrates that the system can detect later modifications of locally stored sensor readings.

8.4 Backend Service Results

The backend scripts were configured as systemd services. This includes the synchronization service, the blockchain anchoring service and the HTTP provider service.

The services run continuously in the background and can be started automatically by the operating system. This improves the reliability of the prototype because the main backend components do not depend on manual terminal execution.

8.5 Mobile Application Results

The project also produced a functional Flutter mobile application. The application connects to the Python HTTP provider and displays live temperature data and historical readings.

This confirms that the telemetry data can be accessed not only through Grafana, but also through a dedicated mobile interface. The mobile application provides a simple and user-friendly way to monitor the system.

The implemented mobile access flow is:

InfluxDB → HTTP Provider → Flutter Mobile Application

8.6 Arrowhead Integration Result

Eclipse Arrowhead was analyzed as a possible service discovery and orchestration framework. The intended architecture was based on the Service Registry, Orchestrator and Authorization systems.

The complete Arrowhead Core integration was not deployed in the final prototype. The main reason was the incompatibility between the Arrowhead 4.x API used by the reference implementation and the readily available Docker images based on Arrowhead 5.x. The compatible 4.6.2 version required manual compilation and additional configuration for the database, certificates and core services.

As a result, the project kept the HTTP provider functional as a direct backend service and documented the Arrowhead-based integration as future work.

8.7 Final Result Summary

The final implementation includes the following completed components:

- a simulated IoT temperature sensor;
- MQTT-based data transmission;
- Node-RED data processing flows;
- InfluxDB time-series storage;
- Grafana real-time visualization;
- local JSONL data persistence;
- automatic synchronization with InfluxDB;
- blockchain anchoring with Ganache, Solidity and Web3.py;
- integrity verification using locally recalculated hashes;
- systemd services for continuous background execution;
- a Python HTTP provider;
- a Flutter mobile application for live and historical monitoring.

Overall, the project achieved its main objective: to build a modular IoT telemetry platform with local resilience, visualization, blockchain-based integrity verification and mobile access. The only major component not fully deployed was the complete Arrowhead Core integration, which remains a future improvement.

9 Limitations and Future Work

Although the project achieved its main objectives, the final implementation remains a prototype and has several limitations. These limitations are important because they indicate the difference between a functional demonstration and a production-ready IoT platform.

9.1 Limitations

9.1.1 Simulated IoT Device

The current implementation uses a Node-RED simulator instead of a physical IoT device. The simulator generates periodic temperature readings and is useful for testing the complete data flow, but it does not include real hardware constraints such as sensor calibration, power consumption, unstable wireless connectivity or device-level failures.

A future version of the project should replace the simulator with a physical device, such as an ESP32 or Raspberry Pi, connected to a real temperature sensor.

9.1.2 Local Development Blockchain

The blockchain component was implemented using Ganache, which is a local Ethereum development blockchain. This is suitable for testing smart contracts and demonstrating the integrity verification mechanism, but it does not provide the same level of decentralization and persistence as a public or production blockchain network.

If Ganache is restarted without persistent storage, the deployed contracts and stored hashes may be lost. Therefore, the current blockchain implementation should be considered a demonstration of the anchoring concept rather than a production-grade immutable storage layer.

9.1.3 Incomplete Arrowhead Core Integration

Eclipse Arrowhead was analyzed as a possible solution for service discovery, orchestration and authorization. However, the complete Arrowhead Core integration was not deployed in the final prototype.

The main limitation was the incompatibility between the Arrowhead 4.x API used by the reference implementation and the readily available Docker images based on Arrowhead 5.x. The compatible 4.6.2 version required manual compilation, database initialization and certificate configuration.

Therefore, the HTTP provider remained functional as a direct backend service, while the complete Arrowhead-based discovery and orchestration flow was documented as future work.

9.1.4 Limited Security Mechanisms

The current prototype does not implement advanced security mechanisms such as authentication, encrypted communication, user roles or secure API access. The HTTP provider exposes data through simple endpoints, which is acceptable for a local prototype but not sufficient for a production deployment.

A production-ready version should include HTTPS, authentication tokens, access control and secure management of configuration files and credentials.

9.1.5 Limited Mobile Features

The Flutter application provides live and historical temperature monitoring, but it does not yet include advanced features such as push notifications, user authentication, automatic backend discovery or complex alert configuration.

The application is functional for monitoring purposes, but it can be extended with more user-oriented features.

9.2 Future Work

Several improvements can be added in future versions of the project.

- Replace the Node-RED simulator with a physical IoT device, such as an ESP32 or Raspberry Pi, connected to a real temperature sensor.
- Add support for multiple sensors and multiple device identifiers.
- Deploy the blockchain component on a persistent private blockchain or public test network.
- Add persistent storage for Ganache or replace it with a more production-like blockchain environment.
- Complete the Arrowhead Core integration by deploying the Service Registry, Orchestrator and Authorization systems.
- Replace hardcoded backend addresses with dynamic service discovery or configurable application settings.
- Add authentication and HTTPS support for the HTTP provider.
- Implement mobile push notifications for threshold-based alerts.
- Improve the Flutter application with settings, device selection and configurable refresh intervals.
- Add a chatbot or voice assistant capable of querying the latest sensor values and system status.
- Extend the Grafana dashboard with more panels, alert rules and long-term analytics.

9.3 Future Architecture Improvement

A more advanced version of the system could use Arrowhead to remove the dependency on hardcoded IP addresses. In this improved architecture, the HTTP provider would register itself in the Arrowhead Service Registry, while the mobile application would request the temperature service through the Orchestrator.

This would make the system more flexible, because the provider address could change without requiring modifications in the mobile application. It would also allow better access control through the Authorization system.

9.4 Summary

The current implementation demonstrates a complete functional prototype, but several components can be improved for production use. The most important future improvements are the use of a real IoT device, persistent blockchain deployment, complete Arrowhead integration, dynamic service discovery, stronger security and enhanced mobile features.

10 Conclusion

This project presented the design and implementation of a modular IoT telemetry platform for temperature monitoring. The system integrates several technologies in order to provide data generation, communication, processing, storage, visualization, local resilience, blockchain-based integrity verification and mobile access.

The main objective of the project was achieved by implementing an end-to-end data flow. Temperature readings are generated by a simulated IoT device, transmitted through MQTT, processed using Node-RED, stored in InfluxDB and visualized in Grafana. This confirmed that the core telemetry pipeline is functional and that sensor data can be collected and monitored in real time.

A local resilience mechanism was also implemented using a JSONL storage file and a Python synchronization service. This mechanism allows the system to preserve data locally and synchronize it with InfluxDB when the database becomes available. By running the synchronization script as a systemd service, the platform became more reliable and less dependent on manual execution.

The blockchain component demonstrated how data integrity can be verified without storing raw sensor data on-chain. Sensor readings were grouped into batches, hashed using Keccak-256 and anchored in a Solidity smart contract deployed on Ganache. The verification script confirmed that the locally stored data could be compared with the blockchain hash in order to detect later modifications.

The project also included a Flutter mobile application connected to a Python HTTP provider. This allowed live and historical temperature values to be accessed from an Android device, providing a more user-friendly monitoring interface than terminal commands or direct database access.

Eclipse Arrowhead was analyzed as a possible service discovery and orchestration framework. Although the complete Arrowhead Core integration was not finalized due to version incompatibilities and configuration complexity, the project documented the intended architecture and identified how Arrowhead could solve the problem of hardcoded or non-persistent backend addresses.

Overall, the project resulted in a functional IoT telemetry prototype and demonstrated how multiple modern technologies can be integrated into a single system. The main remaining improvements are the use of a real physical IoT device, persistent blockchain deployment, complete Arrowhead integration, stronger security mechanisms and enhanced mobile features such as notifications and configurable settings.

11 References

- https://github.com/beia/AHT_wp4_toolchain_demo
- https://docs.google.com/document/d/1Ua652-jsFVKE4_eBWP_wFqscUYVFvfnwBncbR3nfmyo/edit?tab=t.0#heading=h.2sacf2g8lt2x
- <https://chatgpt.com/>
- <https://mqtt.org/>
- <https://www.blockchain-council.org/blockchain/ganache-blockchain-all-you-need-to>
- https://www.youtube.com/watch?v=N0g_gMo_Y4o&list=PLr0AcmG40l03Xjf8RbXgIN1sM1b2TVX

- <https://nodered.org/docs/getting-started/docker>
- <https://grafana.com/docs/>
- <https://docs.flutter.dev/learn/pathway>